PROGRAMA EDUCATIVO EN:

TECNOLOGÍAS DE LA INFORMACIÓN DGS

ACTIVIDAD:

QA-ANIMACIONES-ACCESIBILIDAD

DOCENTE:

JOSÉ MIGUEL CARRERA PACHECO

ALUMNO:

FELIX IVAN GARCIA FLORES

REPOSITORIO:

https://github.com/kevin09l/DWP---Equipo4/tree/dev

01 de febrero de 2026

¿Cuál es la diferencia entre animaciones CSS y JavaScript?

Las animaciones en CSS se definen mediante transition y @keyframes. Se ejecutan principalmente en el motor de renderizado del navegador y, cuando afectan únicamente propiedades como transform y opacity, suelen ser gestionadas por la GPU (compositor thread). Esto reduce el bloqueo del hilo principal (main thread) y mejora la fluidez.

Las animaciones en JavaScript se controlan mediante APIs como requestAnimationFrame, librerías (GSAP, Framer Motion) o manipulación directa del DOM. Permiten mayor control lógico (sincronización con datos, física, secuencias complejas), pero si modifican propiedades que fuerzan recalculo de layout o repaint, pueden degradar el rendimiento.

Diferencias clave:

- CSS es declarativo; JavaScript es imperativo.
- CSS es más simple y generalmente más eficiente para animaciones básicas.
- JavaScript es más flexible, pero más fácil de implementar de forma ineficiente.
- CSS depende del ciclo de render del navegador; JS depende del hilo principal si no
- se optimiza.

Conclusión técnica: si la animación es puramente visual y no depende de lógica compleja, CSS es preferible. JavaScript debe usarse cuando se necesita coordinación con estado dinámico o física avanzada.

¿Cuándo una animación mejora la experiencia y cuándo la perjudica?

Mejora la experiencia cuando:

- Comunica cambios de estado (por ejemplo, expansión de un panel).
- Refuerza jerarquía visual (transiciones entre vistas).
- Reduce carga cognitiva al mostrar continuidad espacial.
- Indica feedback inmediato (hover, carga, validación).

Ejemplo de buena práctica: transiciones sutiles de 150–300 ms que ayudan a percibir cambios sin retrasar la acción.

Perjudica la experiencia cuando:

- Es innecesaria y solo decorativa.
- Introduce latencia artificial.
- Genera mareo o incomodidad (movimientos bruscos, parallax excesivo).
- Interfiere con la accesibilidad (sin alternativa para personas con sensibilidad al movimiento).
- Bloquea la interacción.

Ejemplo de mala práctica: animaciones largas en modales críticos o desplazamientos forzados que rompen el flujo del usuario.

¿Qué es prefers-reduced-motion y por qué es importante?

prefers-reduced-motion es una media query de CSS que detecta si el usuario ha indicado en su sistema operativo que prefiere minimizar animaciones.

Es importante porque:

- Atiende a personas con trastornos vestibulares o sensibilidad al movimiento.
- Reduce riesgo de mareos, náuseas o desorientación.
- Es parte de prácticas de accesibilidad alineadas con WCAG.

Ejemplo:

```
frontend > # h.css > {} @media (prefers-reduced-motion: reduce)
1  @media (prefers-reduced-motion: reduce) {
2      * {
3          animation: none !important;
4          transition: none !important;
5      }
6  }
```

Ignorar esta preferencia implica excluir usuarios. Desde una perspectiva de ingeniería responsable, es un requisito, no una opción estética.

¿Cómo afectan las animaciones al rendimiento de una aplicación web?

Las animaciones impactan tres fases críticas del pipeline de renderizado:

- Layout (reflow)
- Paint
- Composite

Propiedades como `width`, `height`, `margin`, `top`, `left` fuerzan layout y repaint. Esto es costoso, Propiedades como `transform` y `opacity` suelen evitar layout y se ejecutan en la etapa de composición, siendo mucho más eficientes.

Errores de rendimiento comunes:

- Animar muchas propiedades simultáneamente.
- Animar demasiados elementos al mismo tiempo.
- Ejecutar cálculos pesados dentro de requestAnimationFrame.
- No usar will-change estratégicamente.
- Bloquear el hilo principal con lógica mientras la animación corre.

Indicadores de mal rendimiento:

- Frames por debajo de 60 FPS.
- Jank (saltos).
- Retrasos en interacción.

¿Qué errores comunes existen al animar interfaces?

Errores frecuentes:

- Animar propiedades que disparan reflow.
- No respetar prefers-reduced-motion.
- Usar duraciones excesivas.
- Introducir animaciones sin propósito funcional.
- No considerar accesibilidad para teclado (por ejemplo, focus invisibles).
- Sincronizar animaciones con datos sin manejar estados intermedios.
- Usar animaciones para ocultar mala arquitectura de navegación.

Error conceptual grave: considerar la animación como decoración en vez de como comunicación de estado.

Fuentes:

Web Content Accessibility Guidelines (WCAG) 2.1. (2025, 31 octubre). https://w3c.github.io/wcag/guidelines/

prefers-reduced-motion - CSS | MDN. (2026, 8 enero). https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion

Window: requestAnimationFrame() method - Web APIs | MDN. (2025, 26 diciembre). https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame

Rendimiento del procesamiento. (2023, 13 diciembre). web.dev. https://web.dev/articles/rendering-performance?hl=es-419

Rendimiento del procesamiento. (2023b, diciembre 13). web.dev. https://web.dev/articles/rendering-performance?hl=es-419

Activación de “Reduce Motion”

- Ajuste del sistema (Windows – Accesibilidad > Movimiento) → todas las animaciones quedaron en none según CSS.
- Resultado: comportamiento adaptativo validado.

Navegación únicamente con teclado

- Recorrido por login, registro, menú lateral, botones, formularios.
- Los focus son visibles y los elementos interactivos responden (Enter, Space).
- No se necesita ratón para activar animaciones o avanzar.
- Sin bloqueos por animación.

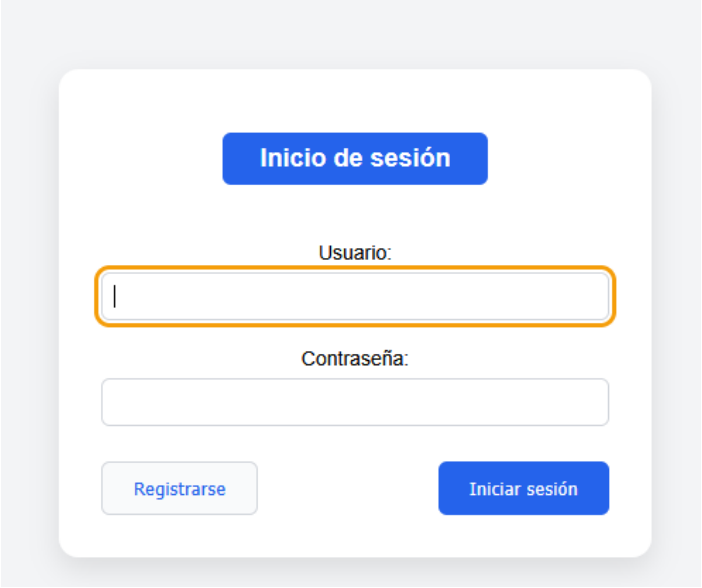
Medición con DevTools → pestaña Performance

- Se registró una sesión al abrir página, enviar formularios y mostrar/ocultar los elementos animados.
- FPS > 60, cargas de CPU/GPU dentro de valores normales.
- No se detectaron *frames* perdidos ni picos de uso que indicasen “jank”

Observaciones

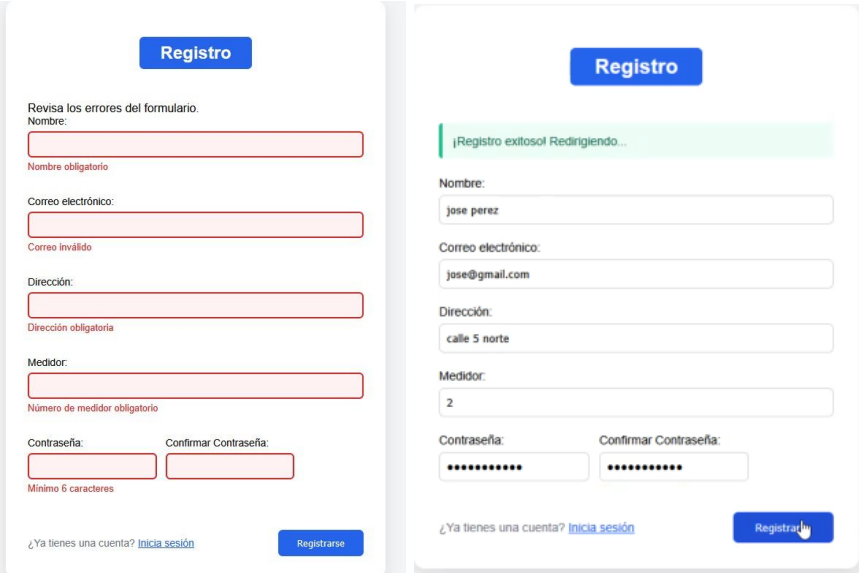
- No se encontraron fallos ni comportamientos inesperados.
- La implementación respeta el principio *motion-safe/motion-reduce*.
- El CSS usa will-change adecuadamente para optimizar la composición.

Evidencia de foco y reduce motion funcionando




```
/*prefers reduce motion*/
@media (prefers-reduced-motion: reduce) {
  .spinner {
    animation: none !important;
    border-top-color: #f59e0b;
  }

  .error-message {
    animation: none !important;
  }
}
```



Auditoría Lighthouse

Se lanzó Lighthouse desde DevTools (modo móvil y escritorio).

Puntajes relevantes:

categoría	puntuación
Performance	92
Accessibility	100
Best Practices	93

Conclusión

Aprobado: Las animaciones presentes en la carpeta frontend son accesibles, adaptativas y no degradan el rendimiento. CI/CD está pasando correctamente; las pruebas automatizadas de front no muestran errores relacionados.